



UNIVERSIDAD PRIVADA DE TACNA

FACULTAD DE INGENIERÍA

Escuela Profesional de Ingeniería de Sistemas

**Proyecto DataQuest
Validador de Normalización de Bases de Datos
Relacionales**

Curso: Base de Datos II

Docente: Patrick Jose Cuadros Quiroga

Integrantes:

Dongo Palza, Manuel Andree (2023076842)

Perez Peralta, Fabrizio Salvador (2023077476)

**Tacna - Perú
2026**



CONTROL DE VERSIONES

Versión	Hecha por	Revisada por	Aprobada por	Fecha	Motivo
1.0	DPM / PPF	Equipo DataQuest	Docente	04/07/2026	Versión inicial del FD04/SAD para DataQuest.

DataQuest: Validador de Normalización de Bases de Datos Relacionales Documento de Arquitectura de Software Versión 1.0



Contenido

CONTROL DE VERSIONES	2
I. INTRODUCCIÓN.....	5
A. Propósito	5
B. Alcance	5
C. Definiciones, siglas y abreviaturas	6
D. Referencias	6
E. Visión general.....	6
II. REPRESENTACIÓN ARQUITECTÓNICA	7
A. Modelo 4+1.....	7
B. Estilo arquitectónico.....	7
C. Decisiones arquitectónicas principales	7
D. Restricciones arquitectónicas	7
III. OBJETIVOS Y LIMITACIONES ARQUITECTÓNICAS.....	8
A. Seguridad	8
B. Disponibilidad	8
C. Rendimiento	8
D. Adaptabilidad.....	8
E. Escalabilidad	8
F. Mantenibilidad.....	8
G. Usabilidad	8
H. Interoperabilidad.....	8
IV. ANÁLISIS DE REQUERIMIENTOS	9
A. Requerimientos funcionales priorizados.....	9
B. Requerimientos no funcionales priorizados	9
C. Trazabilidad requerimiento-arquitectura.....	9
V. VISTA DE ESCENARIOS / CASOS DE USO	10
A. Actores	10
B. Casos de uso detallados.....	11
VI. VISTA LÓGICA	11
A. Diagrama contextual.....	11
B. Diagrama de paquetes.....	11
C. Diagrama de clases	12
D. Diagrama de objetos	15



E. Diagramas de secuencia	16
F. Diagrama de comunicación.....	18
VII. VISTA DE PROCESOS.....	19
A. Proceso actual.....	19
B. Proceso propuesto.....	19
C. Diagrama de actividades del flujo de validación.....	19
D. Diagrama de actividades del historial.....	20
E. Diagramas de tiempo / Timing diagrams.....	20
F. Manejo de errores.....	22
VIII. VISTA DE DESARROLLO.....	23
A. Diagrama de capas.....	23
B. Diagrama de componentes.....	23
C. Organización de carpetas.....	24
D. Dependencias técnicas.....	24
E. Contratos entre módulos.....	24
F. API y endpoints.....	24
IX. VISTA FÍSICA / DESPLIEGUE.....	25
A. Infraestructura local	25
B. Infraestructura propuesta.....	25
C. Seguridad de despliegue	26
X. VISTA DE DATOS	26
A. Modelo lógico	26
B. Modelo físico	27
C. JSONB y estructuras flexibles	27
D. Auditoría e historial.....	27
XI. CALIDAD ARQUITECTÓNICA.....	27
A. Observaciones de calidad por componente	28
XII. RIESGOS ARQUITECTÓNICOS.....	28
XIII. CONCLUSIONES ARQUITECTÓNICAS	28
XIV. RECOMENDACIONES	28
ANEXO ÚNICO. FUENTES EDITABLES RESUMIDAS DE DIAGRAMAS	29



I. INTRODUCCIÓN

A. Propósito

El propósito de este Documento de Arquitectura de Software (SAD) es definir de manera precisa la arquitectura del proyecto DataQuest, una plataforma académica orientada al análisis, diagnóstico y mejora de esquemas de bases de datos relacionales. El documento describe cómo se organiza el sistema, cómo se comunican sus componentes, cómo fluye la información desde la interfaz hasta el motor de normalización y cómo se conserva la trazabilidad de cada validación mediante Supabase/PostgreSQL.

La arquitectura documentada se sustenta en el modelo 4+1, porque permite explicar DataQuest desde la perspectiva de escenarios, lógica interna, procesos, desarrollo e infraestructura física. Esta forma de representación es adecuada para demostrar que DataQuest no es únicamente una pantalla de validación, sino una solución con separación de responsabilidades, API, persistencia, seguridad, pruebas y posibilidad de evolución.

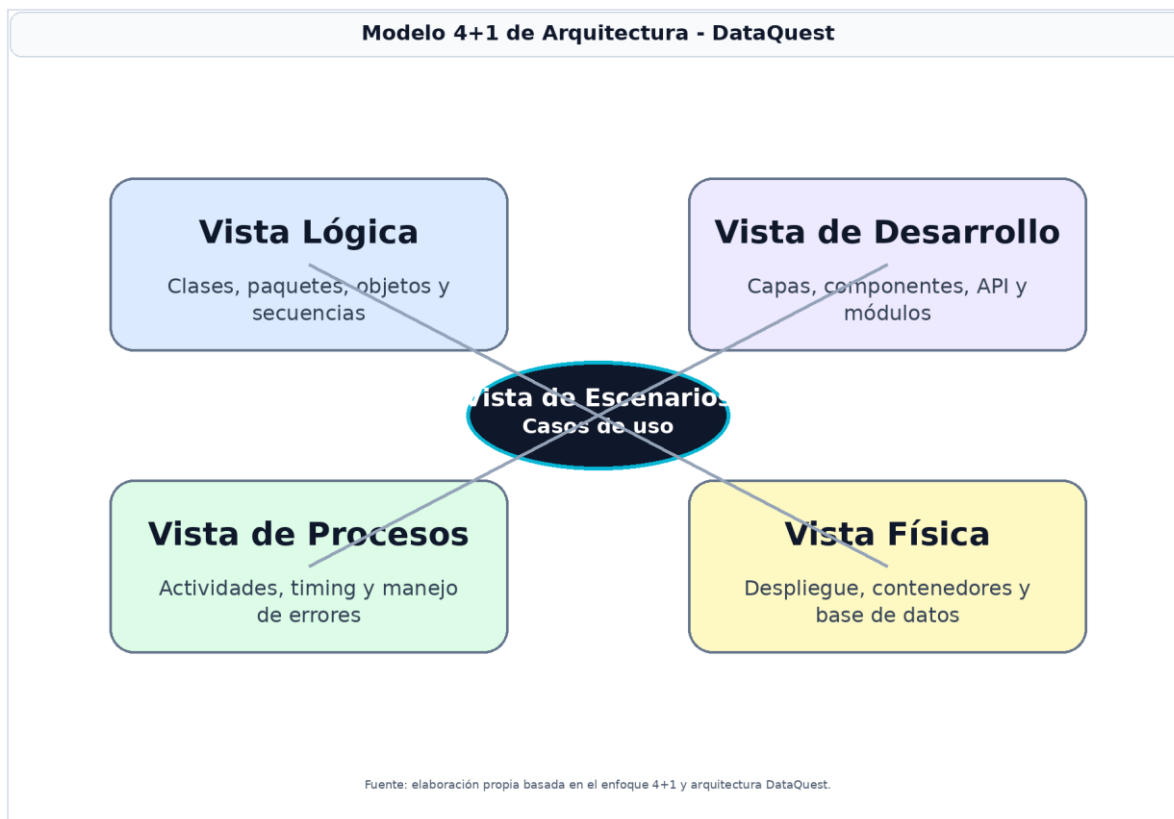


Figura 1. Modelo 4+1 aplicado a DataQuest.

B. Alcance

El alcance de este FD04 comprende la arquitectura del sistema DataQuest en su versión académica actual y en su arquitectura objetivo inmediata. Se documentan los módulos implementados, parcialmente implementados y recomendados sin mezclar contenido de proyectos externos.

- Interfaz web desarrollada con Streamlit y organizada mediante app.py y vistas independientes.



- Motor core en Python para parsing, diagnóstico, validación 1FN, 2FN y 3FN, generación de sugerencias y corrección asistida.
- API REST con FastAPI, incluyendo el endpoint POST /api/normalizacion.
- Persistencia en Supabase/PostgreSQL para usuarios, perfiles, presencia e historial de validaciones.
- Extensión de Visual Studio Code y CLI para ejecutar validaciones desde el editor o desde línea de comandos.
- Pruebas unitarias, integración, UI, BDD, workflows de GitHub Actions, Docker y Terraform como soporte DevOps.

Fuera del alcance: DataQuest no reemplaza la evaluación formal del docente, no garantiza detección perfecta de todas las dependencias funcionales si el usuario no provee información suficiente y no debe utilizarse como único mecanismo certificador en entornos productivos críticos.

C. Definiciones, siglas y abreviaturas

Término	Definición aplicada a DataQuest
SAD	Software Architecture Document. Documento que describe vistas, decisiones, restricciones y calidad arquitectónica.
SRS	Software Requirements Specification. Documento de requerimientos que alimenta este FD04.
UML	Lenguaje de modelado usado para casos de uso, clases, secuencias, actividades, despliegue y datos.
1FN	Primera Forma Normal: exige clave primaria y evita grupos repetitivos o atributos multivaluados.
2FN	Segunda Forma Normal: elimina dependencias parciales respecto a claves compuestas.
3FN	Tercera Forma Normal: elimina dependencias transitivas entre atributos no clave.
FastAPI	Framework Python usado por api.py para exponer el endpoint de normalización.
Streamlit	Framework Python utilizado como interfaz web interactiva.
JSONB	Tipo PostgreSQL usado para almacenar estructuras dinámicas de esquemas, violaciones y sugerencias.
RLS	Row Level Security. Política de seguridad recomendada/implementada en Supabase para aislar datos por usuario.

D. Referencias

- FD03/SRS DataQuest: especificación de requerimientos funcionales y no funcionales.
- FD05/Informe Final DataQuest: consolidación del proyecto, tecnología, QA y evidencias.
- FD06/Propuesta de Proyecto DataQuest: justificación, alcance, presupuesto, factibilidad y monitoreo.
- Repositorio DataQuest: app.py, api.py, core/, controllers/, views/, db/, visualizacion/, tests/, vscode-extension/, Dockerfile, Terraform y workflows.
- Documentación oficial de Python, Streamlit, FastAPI, PostgreSQL, Supabase, Graphviz y Visual Studio Code Extension API.
- Pressman y Sommerville como referencias generales de ingeniería y arquitectura de software.

E. Visión general

El documento está organizado de forma progresiva. Primero define el propósito y el alcance arquitectónico; luego presenta el modelo 4+1, las restricciones, los requerimientos priorizados y las vistas necesarias para comprender DataQuest desde escenarios, clases, procesos, componentes, despliegue y datos. Finalmente, desarrolla escenarios de calidad, riesgos, conclusiones, recomendaciones y checklist final de entrega.



II. REPRESENTACIÓN ARQUITECTÓNICA

A. Modelo 4+1

La vista de escenarios funciona como eje integrador, porque representa la interacción de estudiantes, docentes, administradores y consumidores de API con las capacidades del sistema. La vista lógica se enfoca en clases, objetos, paquetes y secuencias. La vista de procesos explica el flujo dinámico de validación. La vista de desarrollo documenta capas, componentes y organización del repositorio. La vista física muestra el despliegue en cliente, servicios Python y Supabase/PostgreSQL.

B. Estilo arquitectónico

DataQuest adopta una arquitectura modular de tipo MVC extendido, con separación clara entre presentación, controladores, dominio, persistencia e integraciones. La interfaz Streamlit actúa como capa de presentación; FastAPI expone el motor mediante HTTP/JSON; el core Python concentra las reglas de normalización; Supabase/PostgreSQL almacena perfiles, presencia e historial; la extensión VS Code y CLI reutilizan el mismo núcleo de análisis.

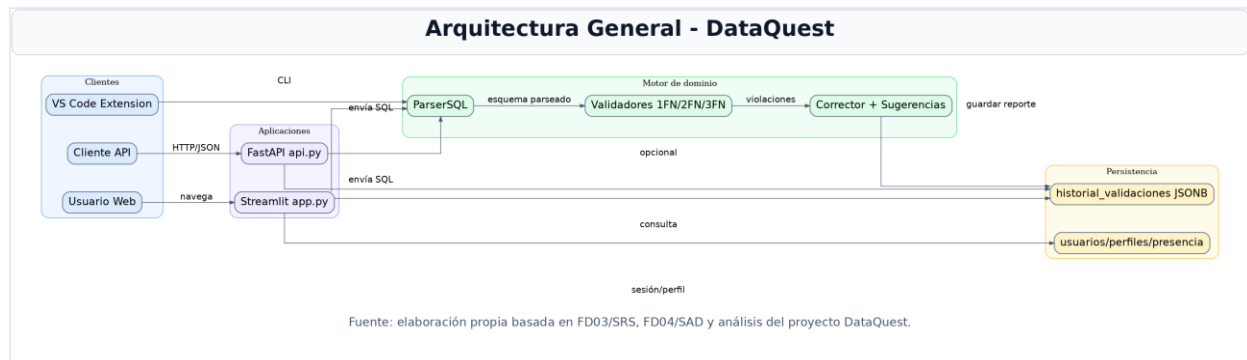


Figura 2. Arquitectura general de DataQuest.

C. Decisiones arquitectónicas principales

Decisión	Justificación	Impacto
Separar Streamlit, API y core	Permite usar el motor desde web, API, CLI y VS Code sin duplicar lógica.	Alta mantenibilidad e interoperabilidad.
Usar JSONB en historial_validaciones	Los esquemas y resultados son variables; JSONB evita una sobre-normalización prematura.	Flexibilidad, pero requiere validaciones desde aplicación.
Usar validadores independientes 1FN, 2FN y 3FN	Cada forma normal tiene reglas y violaciones diferenciadas.	Mejor extensibilidad para BCNF o reglas futuras.
Exponer POST /api/normalizacion	Permite integración con otros clientes y automatización.	Aumenta valor del proyecto más allá de Streamlit.
Mantener pruebas automatizadas	El motor de normalización debe ser verificable y repetible.	Reduce regresiones y mejora confianza académica.

D. Restricciones arquitectónicas

- El análisis de dependencias funcionales combina reglas clásicas y heurísticas; su precisión depende de la calidad del SQL y de las dependencias ingresadas.
- El endpoint api.py permite CORS abierto en desarrollo; para producción académica debe restringirse.
- El script db/modelos.sql usa Supabase/Auth y RLS; la variante solicitada por el usuario considera además public.usuarios como tabla de identidad documentada.



- La extensión VS Code depende de Python disponible en el entorno local donde se ejecuta el CLI.
- CSV y XLS están considerados en el alcance de entrada, pero el parser CSV se identifica como parcial/pendiente según la evidencia del código.

III. OBJETIVOS Y LIMITACIONES ARQUITECTÓNICAS

A. Seguridad

Proteger credenciales, historial, esquemas analizados y resultados de validación. La base documentada usa password_hash, relaciones por user_id y políticas RLS en la implementación Supabase. Se debe evitar guardar contraseñas en texto plano, exponer service role keys, aceptar SQL sin validación o permitir acceso cruzado a historiales.

B. Disponibilidad

Garantizar disponibilidad suficiente para uso académico. El sistema puede ejecutarse localmente con Streamlit y depende de Supabase cuando se consulta o persiste información. Si Supabase no responde, la arquitectura debe mostrar resultados sin perder la entrada del usuario y advertir que no se pudo registrar el historial.

C. Rendimiento

Permitir análisis rápido de esquemas pequeños y medianos. El tiempo objetivo para validaciones ordinarias debe mantenerse en el orden de milisegundos a pocos segundos. Los campos JSONB pueden necesitar índices GIN si crece el historial.

D. Adaptabilidad

La arquitectura debe permitir agregar BCNF, más reglas semánticas, nuevos formatos de entrada y nuevos clientes sin reescribir el core. Para ello se separan parser, diagnóstico, validadores y corrector.

E. Escalabilidad

El crecimiento esperado es académico. La separación de API, core y base permite escalar por módulos. La tabla historial_validaciones debe paginarse e indexarse por user_id y fecha.

F. Mantenibilidad

El código debe conservar módulos pequeños y pruebas. Los cambios al parser o a validadores no deben romper la interfaz ni la API si se mantiene el contrato del servicio.

G. Usabilidad

Los diagnósticos deben ser entendibles para estudiantes. Las violaciones deben indicar forma normal, tabla, columna o dependencia y sugerencia asociada.

H. Interoperabilidad

El sistema debe ser consumible desde Streamlit, FastAPI, CLI, VS Code y potencialmente agentes IA usando JSON estructurado.



IV. ANÁLISIS DE REQUERIMIENTOS

A. Requerimientos funcionales priorizados

Código	Descripción	Prioridad	Componente arquitectónico	Evidencia esperada
RF-01	Ingresar esquema SQL	Alta	Streamlit / API	Entrada SQL centralizada
RF-02	Analizar sentencias CREATE TABLE	Alta	ParserSQL	parse_sql extrae tablas y campos
RF-03	Identificar tablas y campos	Alta	core/parser.py	Estructura tablas[]
RF-04	Detectar claves primarias	Alta	ParserSQL / Validador1FN	pks detectadas
RF-05	Detectar claves foráneas	Media	ParserSQL	fks detectadas
RF-06	Validar 1FN	Alta	validador_1fn.py	Violaciones sin PK o grupos repetitivos
RF-07	Validar 2FN	Alta	validador_2fn.py	Dependencias parciales
RF-08	Validar 3FN	Alta	validador_3fn.py	Dependencias transitivas
RF-09	Detectar violaciones	Alta	diagnostico.py	Listas por forma normal
RF-10	Generar sugerencias	Alta	corrector.py	Sugerencias con impacto y confianza
RF-11	Mostrar resultados en Streamlit	Alta	views/05_resultados.py	Vista de resultados
RF-12	Registrar historial	Alta	historial_validaciones	Persistencia JSONB
RF-13	Consumir validación mediante API	Alta	api.py	POST /api/normalizacion
RF-14	Consultar historial	Media	views/06_historial.py	Listado por usuario
RF-15	Gestionar usuarios	Media	auth_controller / Supabase	Registro/login
RF-16	Gestionar perfiles	Media	perfiles	Nombre y avatar
RF-17	Registrar presencia	Media	presencia / realtime	Usuarios conectados
RF-18	Manejar errores de entrada	Alta	api.py / parser	Errores 400 controlados
RF-19	Generar reporte	Media	generador_pdf.py	Reporte PDF si aplica
RF-20	Ejecutar pruebas QA	Alta	tests/ / workflows	pytest + CI

B. Requerimientos no funcionales priorizados

Código	Categoría	Prioridad	Decisión arquitectónica asociada
RNF-01	Seguridad	Alta	Validación de entrada, password_hash, RLS, variables de entorno.
RNF-02	Rendimiento	Alta	Tiempo de respuesta API controlado y parser eficiente.
RNF-03	Disponibilidad	Media	Uso local + manejo de fallas de Supabase.
RNF-04	Usabilidad	Alta	Mensajes claros y resultados pedagógicos.
RNF-05	Mantenibilidad	Alta	Módulos core separados y pruebas.
RNF-06	Escalabilidad	Media	Paginación e índices en historial.
RNF-07	Portabilidad	Media	Docker y ejecución con Python 3.10+.
RNF-08	Trazabilidad	Alta	historial_validaciones con JSONB.
RNF-09	Interoperabilidad	Alta	API, CLI y VS Code Extension.
RNF-10	Calidad académica	Alta	Explicación de 1FN, 2FN y 3FN.
RNF-11	Compatibilidad	Media	Navegador moderno y VS Code.
RNF-12	Documentación	Alta	FD03, FD04, FD05, FD06, README e integración API.

C. Trazabilidad requerimiento-arquitectura

Requerimiento	Elemento arquitectónico	Vista donde se justifica	Evidencia
RF-06/RF-07/RF-08	Validadores 1FN/2FN/3FN	Vista lógica y proceso	validador_1fn.py, validador_2fn.py, validador_3fn.py
RF-13	API REST	Vista de desarrollo y despliegue	api.py, POST /api/normalizacion
RF-12/RNF-08	Historial	Vista de datos	historial_validaciones JSONB
RNF-01	Seguridad	Calidad y datos	password_hash, RLS, validación de entrada
RNF-09	Interoperabilidad	Vista física y desarrollo	Streamlit, API, CLI, VS Code Extension



V. VISTA DE ESCENARIOS / CASOS DE USO

La vista de escenarios describe cómo los actores principales interactúan con DataQuest. El diagrama se construyó para evitar cruces innecesarios y separar usuarios humanos, clientes API e integración VS Code. El límite del sistema se define como “Sistema DataQuest”.

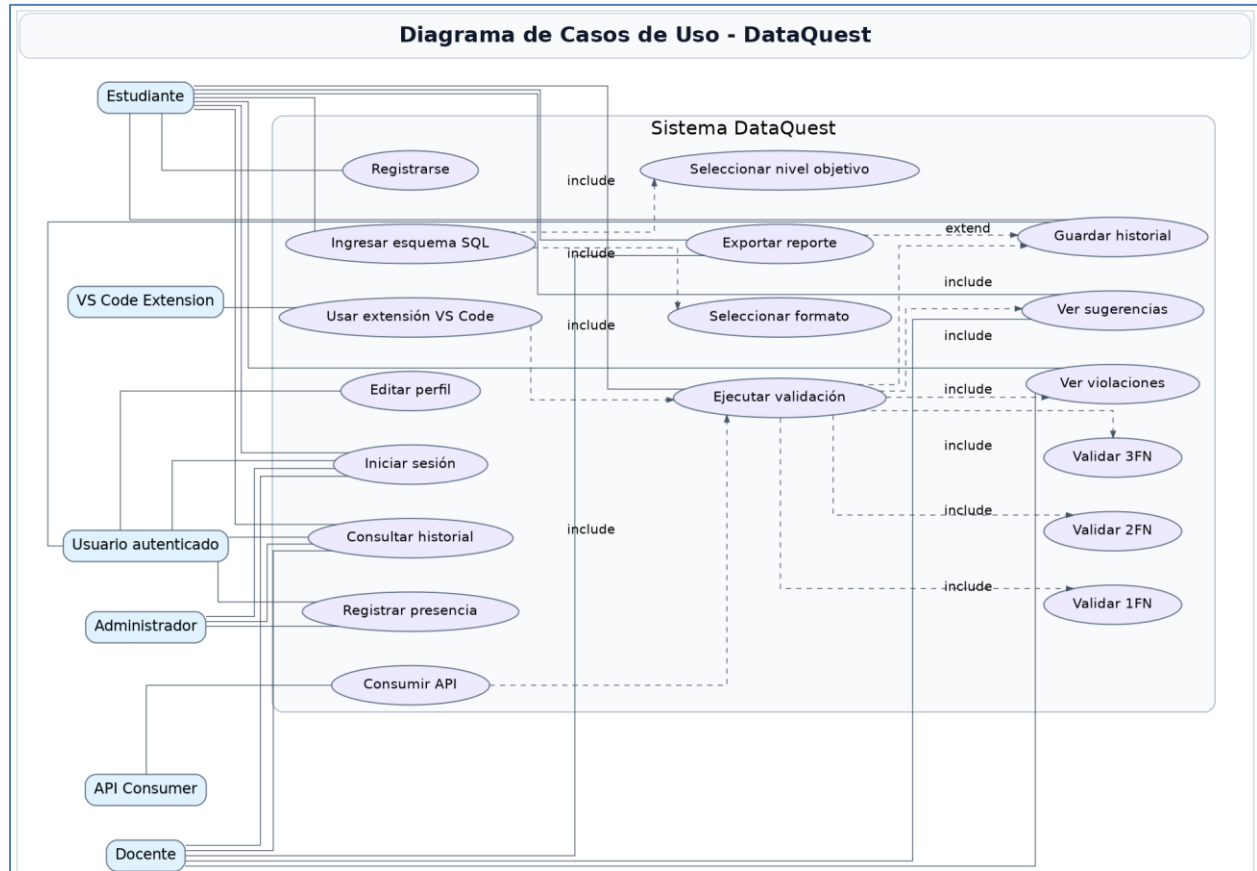


Figura 3. Diagrama de casos de uso de DataQuest.

A. Actores

Actor	Descripción	Responsabilidad
Estudiante	Usuario académico principal.	Ingresa SQL, ejecuta validaciones y revisa sugerencias.
Docente	Usuario evaluador.	Consulta resultados, valida evidencias y puede revisar historiales.
Usuario autenticado	Usuario con sesión activa.	Accede a perfil, historial y presencia.
Administrador	Rol con mayor control académico/técnico.	Supervisa usuarios, actividad y evidencias.
API Consumer	Cliente externo o herramienta automatizada.	Consume POST /api/normalizacion.
VS Code Extension	Cliente IDE del proyecto.	Ejecuta el CLI e inyecta reportes en archivos SQL.



B. Casos de uso detallados

Código	Nombre	Actor	Objetivo	Criterio de aceptación	Req.
CU-04	Ingresar esquema SQL	Estudiante	Capturar CREATE TABLE o texto pegado para análisis.	Debe aceptar SQL no vacío.	RF-01
CU-08	Ejecutar validación	Estudiante	Procesar el esquema y calcular nivel actual.	Incluye CU-09, CU-10 y CU-11.	RF-06/RF-07/RF-08
CU-12	Ver violaciones	Estudiante/Docente	Mostrar incumplimientos por forma normal.	Debe indicar tipo y mensaje.	RF-09
CU-13	Ver sugerencias	Estudiante	Presentar correcciones con impacto y confianza.	Debe orientar al aprendizaje.	RF-10
CU-15	Guardar historial	Usuario autenticado	Persistir análisis en JSONB.	Debe asociarse al user_id.	RF-12
CU-17	Consumir API	API Consumer	Enviar SQL por HTTP/JSON.	Debe responder success/error.	RF-13
CU-23	Usar VS Code Extension	Usuario IDE	Validar desde editor y agregar reporte.	Depende de core/cli.py.	RNF-09

VI. VISTA LÓGICA

A. Diagrama contextual

El diagrama contextual ubica DataQuest frente a sus consumidores y almacenes. Streamlit es la interfaz principal, FastAPI habilita interoperabilidad, el core Python concentra reglas de negocio y Supabase/PostgreSQL conserva identidad, presencia e historial.

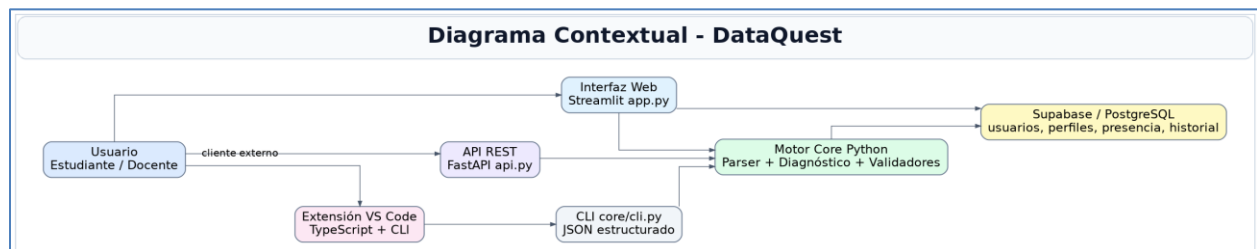


Figura 4. Diagrama contextual de DataQuest.

B. Diagrama de paquetes

Los paquetes reflejan la estructura real del repositorio: controllers/, core/, db/, views/, visualizacion/, vscode-extension/, Dockerfile, Terraform y workflows. Esta organización permite identificar dependencias y responsabilidades.

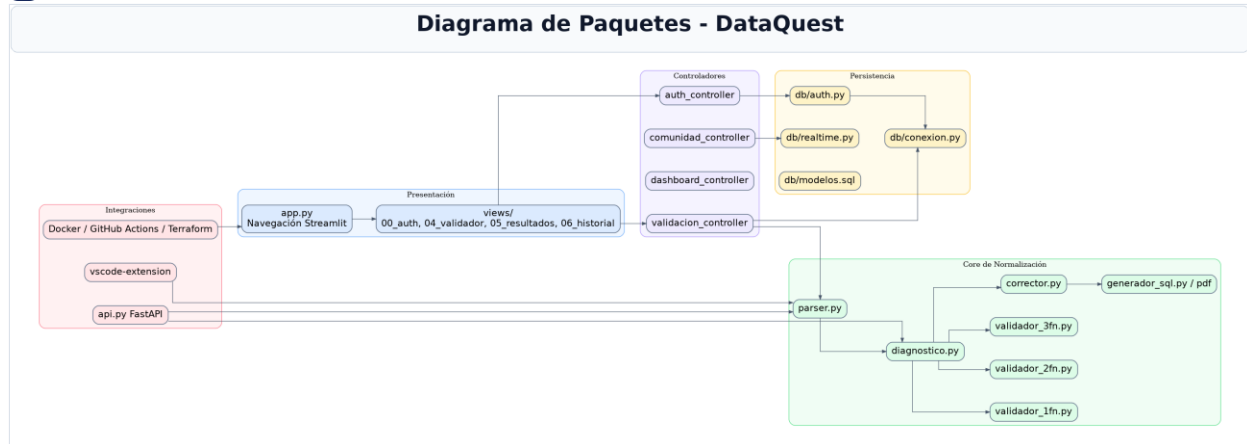


Figura 5. Diagrama de paquetes de DataQuest.

C. Diagrama de clases

El diagrama de clases se construye con dos niveles de precisión: primero, una vista general de arquitectura limpia alineada con los módulos reales; segundo, un detalle del núcleo de normalización. Debido a que el repositorio implementa principalmente funciones Python, varias clases se documentan como clases conceptuales de arquitectura limpia para representar responsabilidades sin afirmar que todas existen literalmente como clases en el código.

Elementos implementados comprobables: `parse_schema`, `parse_sql` y `parse_fds` en `core/parser.py`; `validar_1fn`, `validar_2fn` y `validar_3fn`; `diagnosticar_esquema` e `inferir_dependencias_heuristica`; `generar_sugerencias`; FastAPI con `POST /api/normalizacion`; Streamlit con navegación por vistas; tablas usuarios, perfiles, presencia e historial_validaciones según el esquema de persistencia indicado.

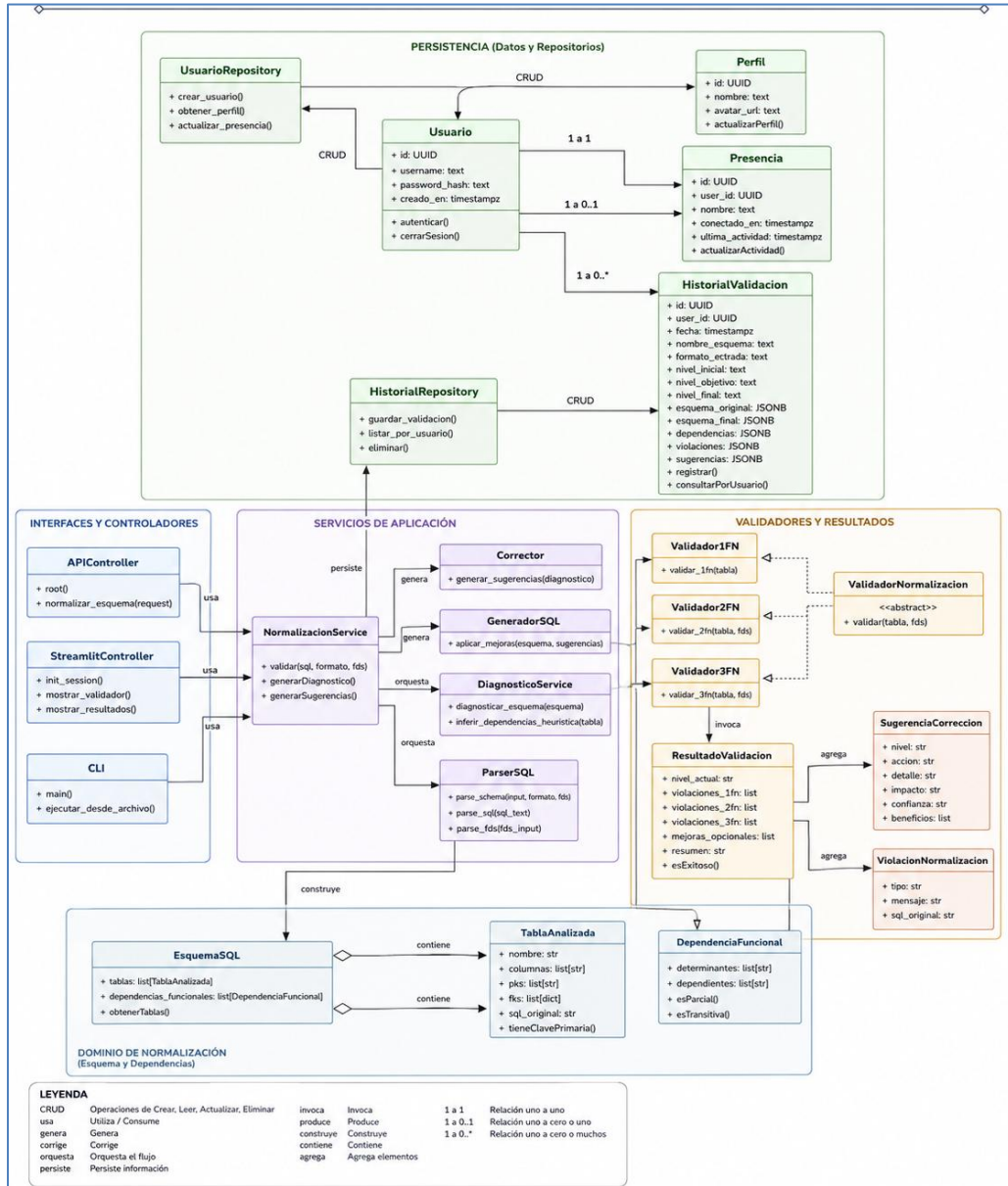


Figura 6. Diagrama de clases preciso de DataQuest.

Precisión del diagrama de clases: Usuario, Perfil, Presencia e HistorialValidacion representan entidades persistentes. ParserSQL, DiagnosticoService, Corrector, GeneradorSQL y NormalizacionService representan servicios de aplicación/dominio. Validador1FN, Validador2FN y Validador3FN especializan la responsabilidad de validación. ResultadoValidacion, ViolacionNormalizacion y SugerenciaCorreccion representan estructuras de salida usadas por Streamlit, FastAPI, CLI y VS Code.



Detalle del Núcleo de Normalización - DataQuest

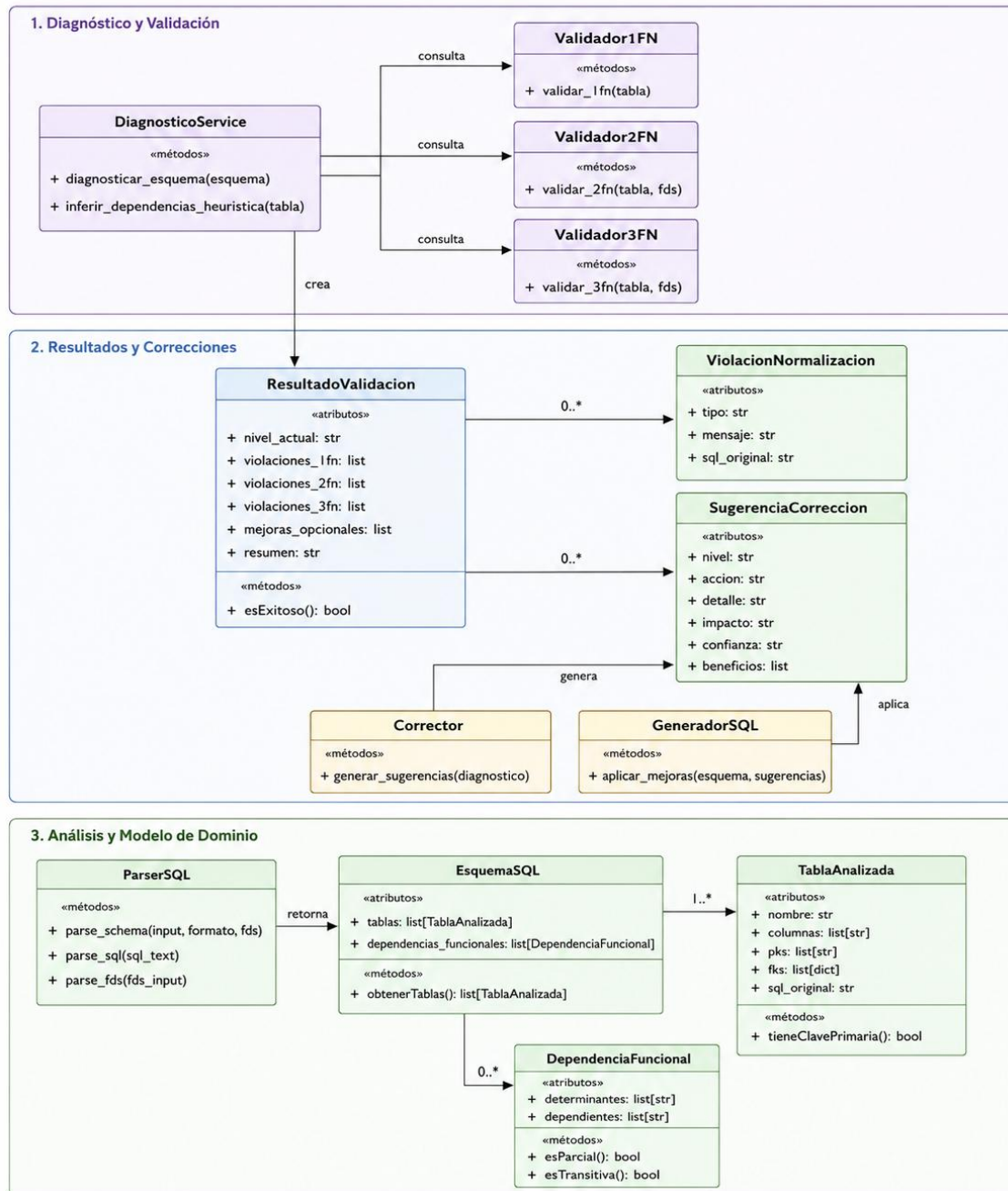


Figura 7. Detalle del núcleo de normalización de DataQuest.



D. Diagrama de objetos

El diagrama de objetos muestra una instancia concreta de ejecución: un usuario autenticado ejecuta una validación del esquema “Sistema biblioteca”, se almacenan datos JSONB de entrada, violaciones y sugerencias.

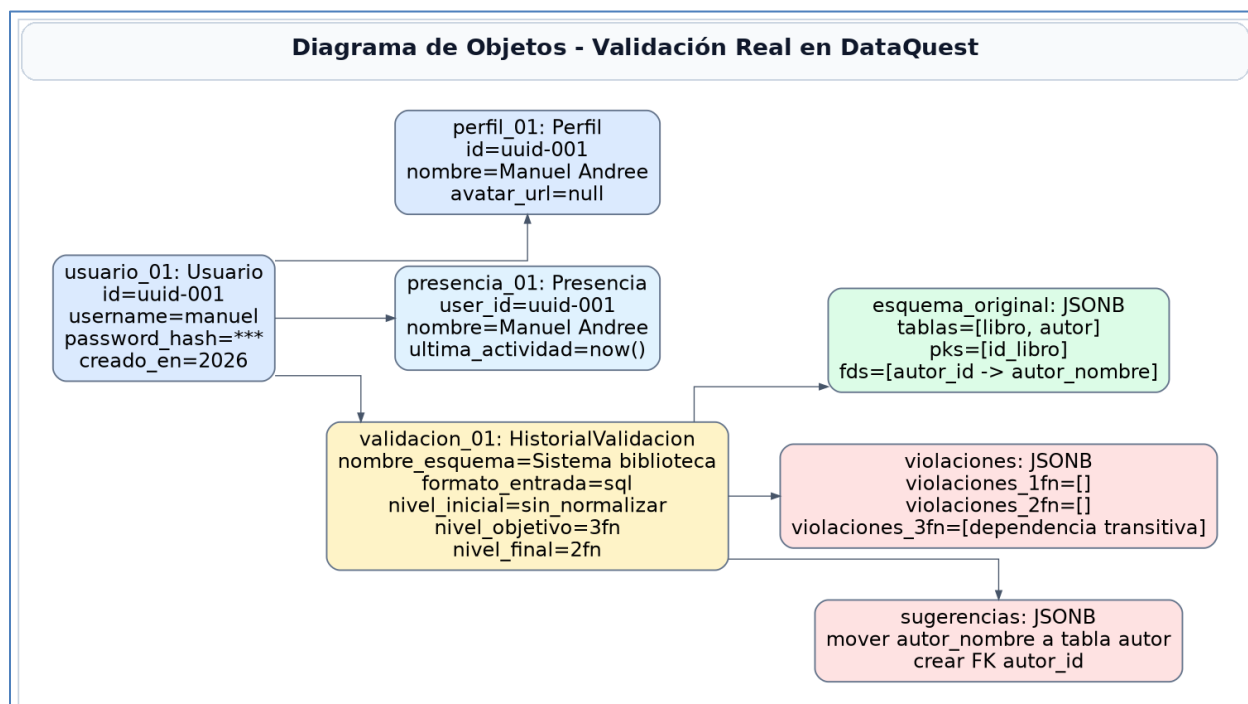


Figura 8. Diagrama de objetos de una validación en DataQuest.



E. Diagramas de secuencia

Se documentan tres secuencias: validación desde Streamlit, consumo externo por API y consulta de historial. Las secuencias incluyen retorno de datos y flujo de persistencia.

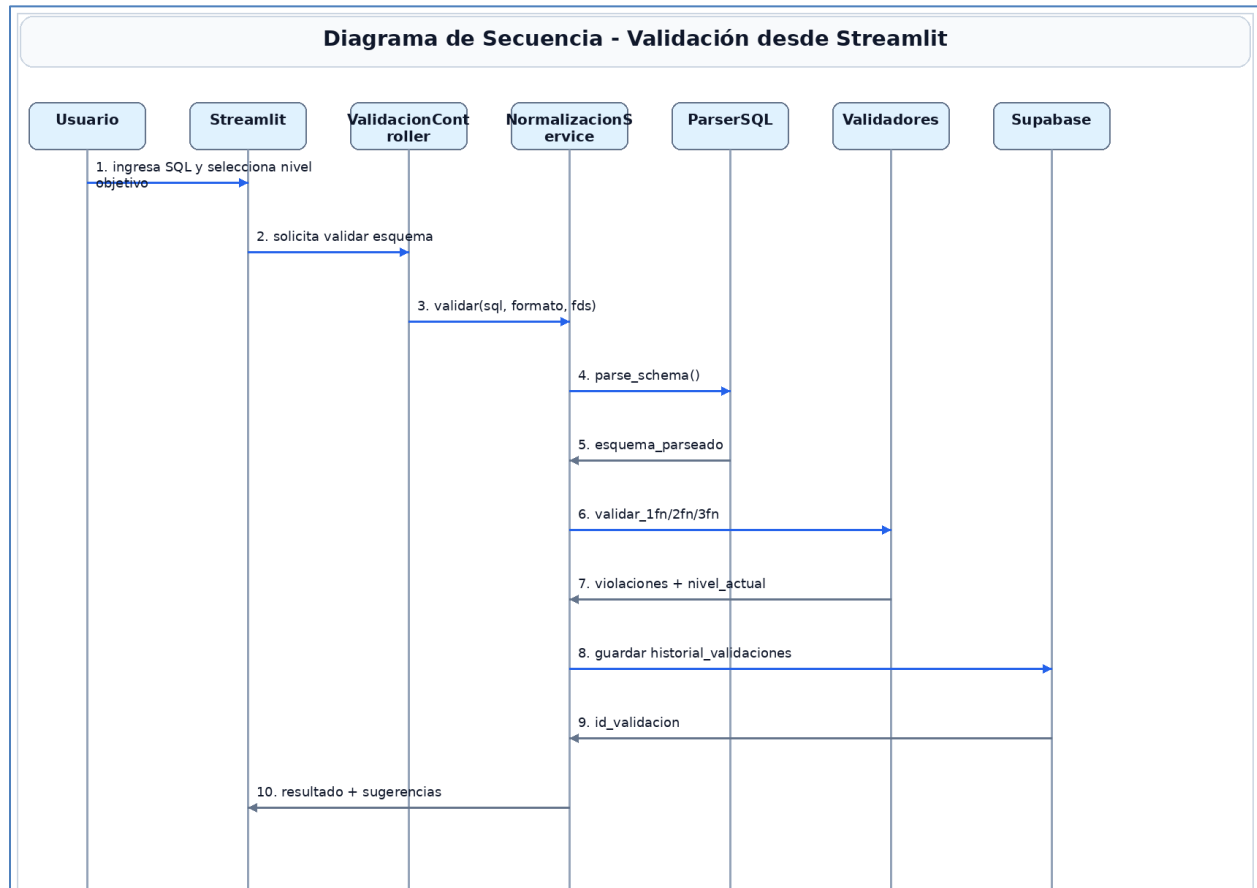


Figura 9. Diagrama de secuencia de validación desde Streamlit.

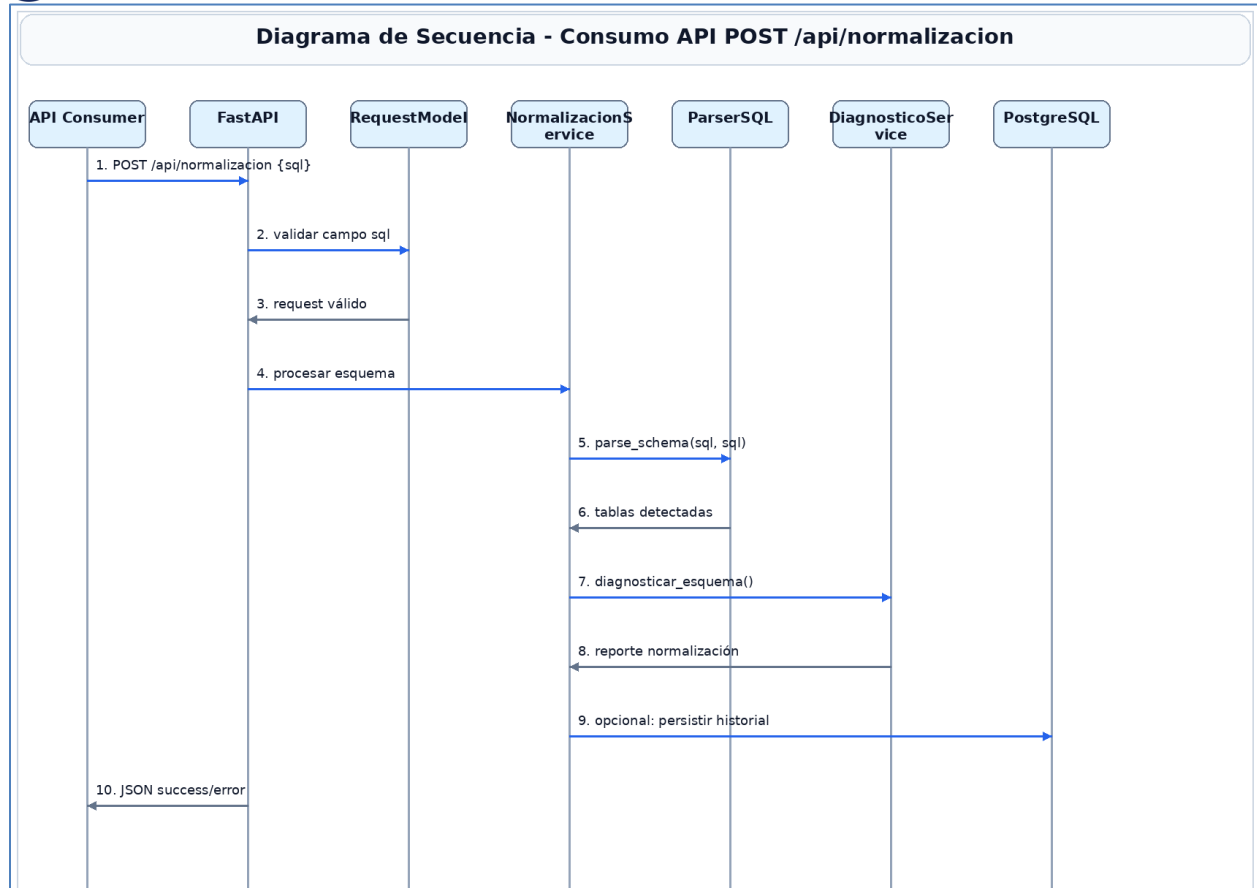


Figura 10. Diagrama de secuencia del endpoint POST /api/normalizacion.

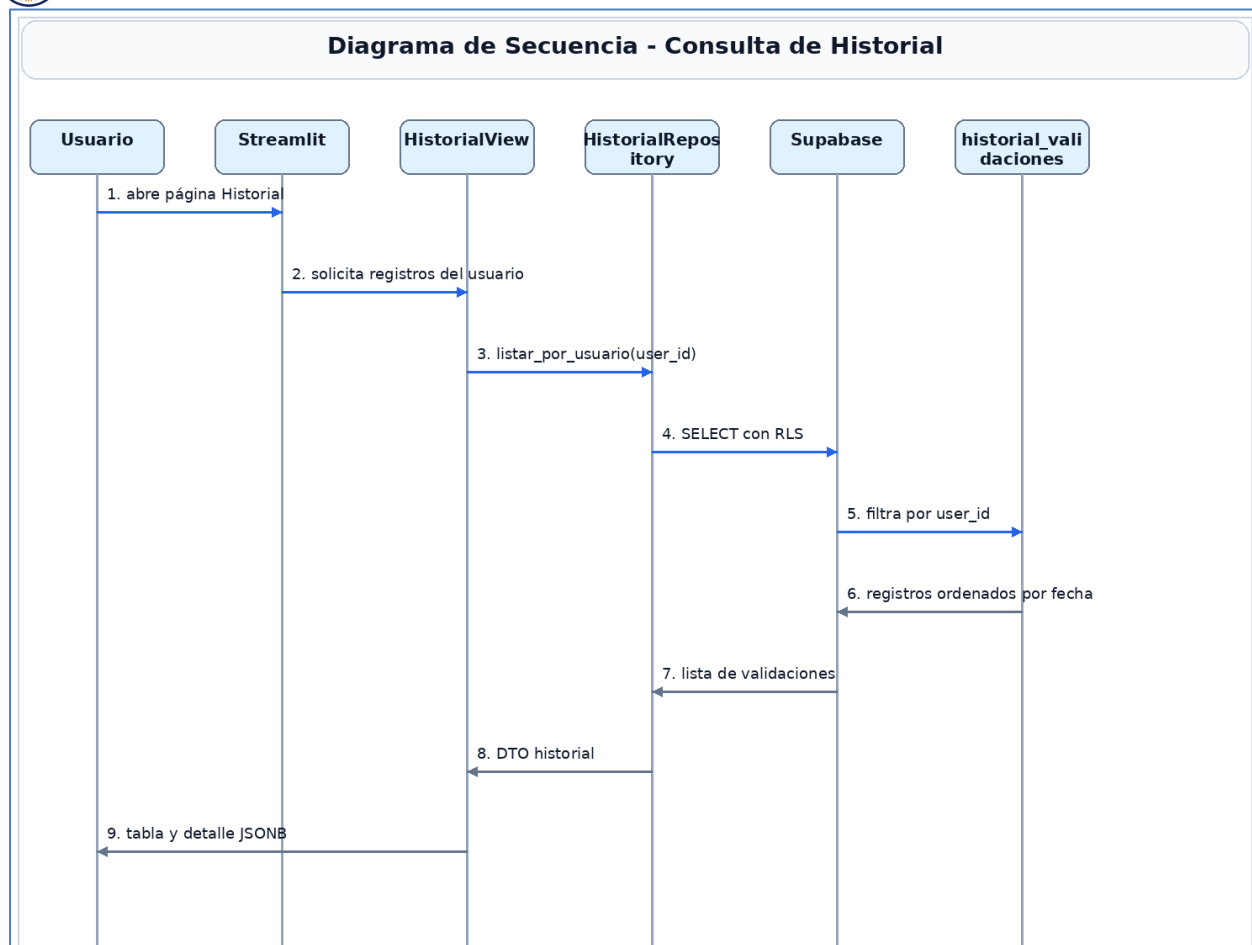


Figura 11. Diagrama de secuencia de consulta de historial.

F. Diagrama de comunicación

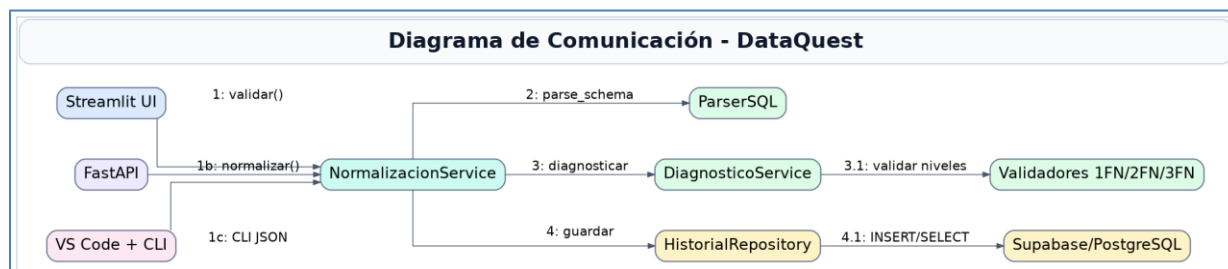


Figura 12. Diagrama de comunicación entre módulos DataQuest.



VII. VISTA DE PROCESOS

A. Proceso actual

Antes de DataQuest, el proceso académico de normalización depende de revisión manual: el estudiante diseña tablas, revisa teoría, intenta detectar anomalías, corrige su esquema y espera retroalimentación docente. Este proceso es lento, subjetivo y poco trazable.

B. Proceso propuesto

Con DataQuest, el usuario ingresa SQL, el parser detecta tablas, el diagnóstico ejecuta validadores 1FN/2FN/3FN, el corrector genera sugerencias, el sistema muestra resultados y, si existe sesión, guarda el análisis en historial_validaciones. El proceso deja evidencia consultable para aprendizaje y sustentación.

C. Diagrama de actividades del flujo de validación

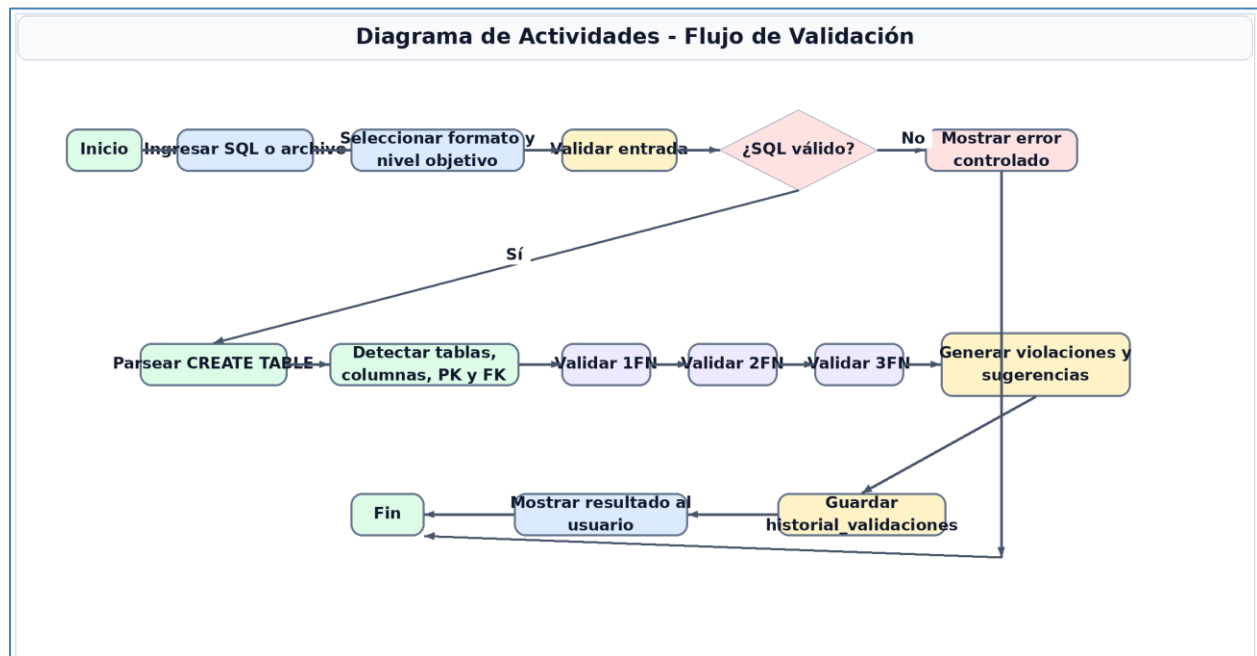


Figura 13. Diagrama de actividades del flujo de validación.



D. Diagrama de actividades del historial

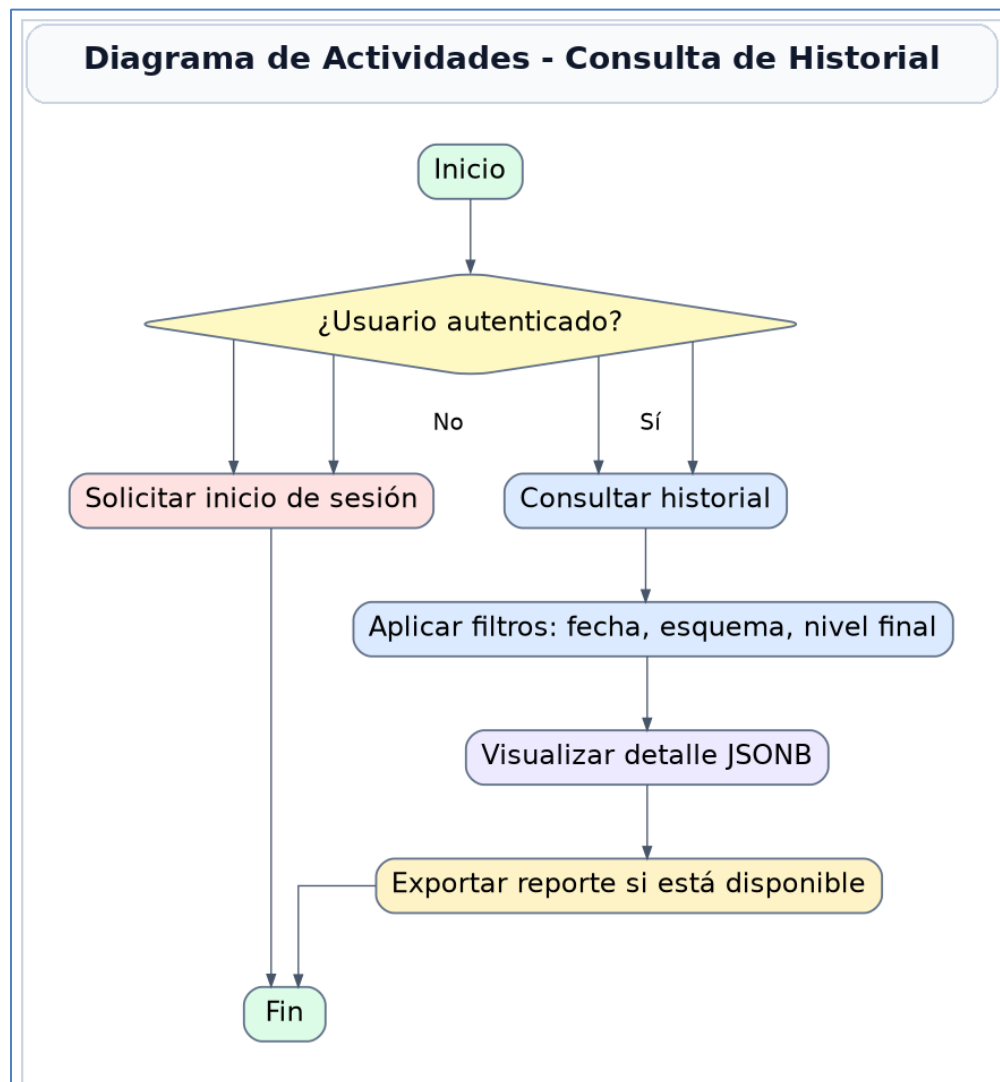


Figura 14. Diagrama de actividades de consulta de historial.

E. Diagramas de tiempo / Timing diagrams

Los diagramas de tiempo muestran estados esperados durante el procesamiento. Los tiempos son metas arquitectónicas referenciales para esquemas pequeños y medianos en entorno académico.

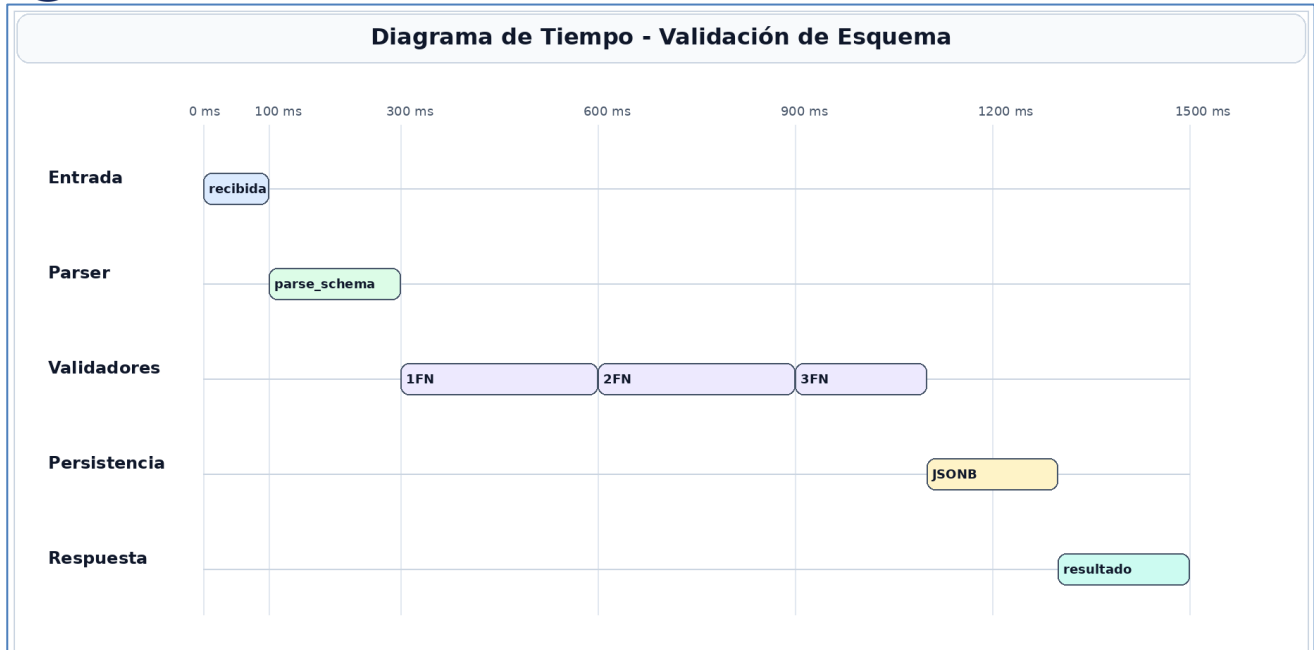


Figura 15. Diagrama de tiempo del procesamiento de validación.

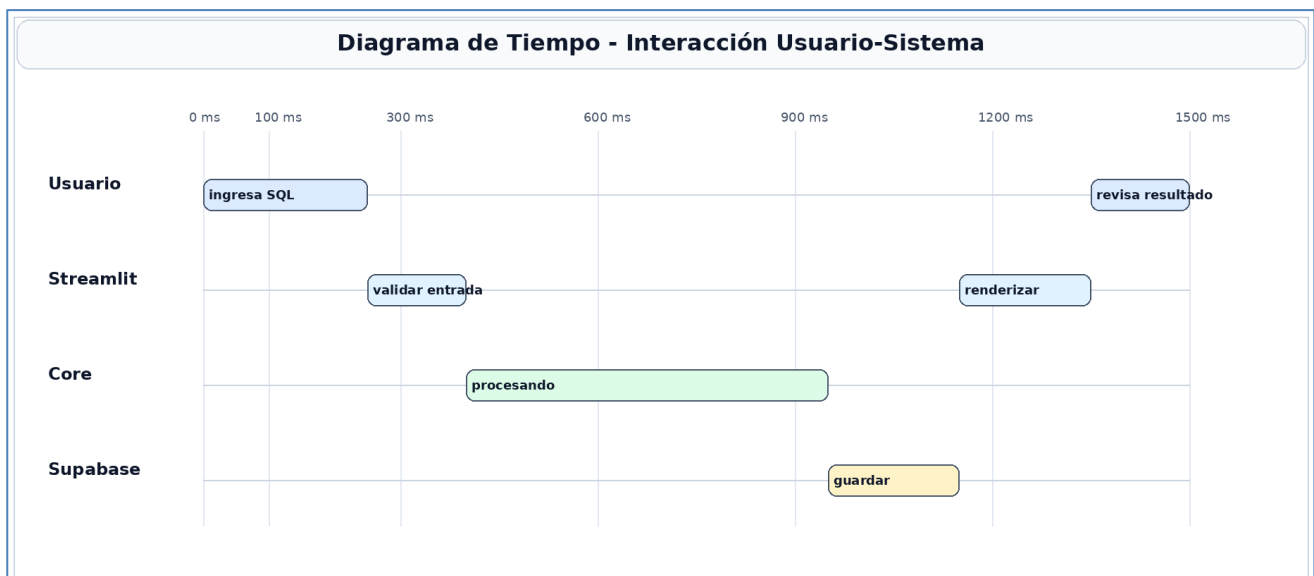


Figura 16. Diagrama de tiempo de interacción usuario-sistema.



F. Manejo de errores

La arquitectura debe responder con errores controlados ante SQL vacío, ausencia de tablas detectables, fallas de parsing o indisponibilidad de Supabase. El endpoint api.py ya devuelve estructuras success/error con códigos 400 y 500 lógicos dentro del JSON.

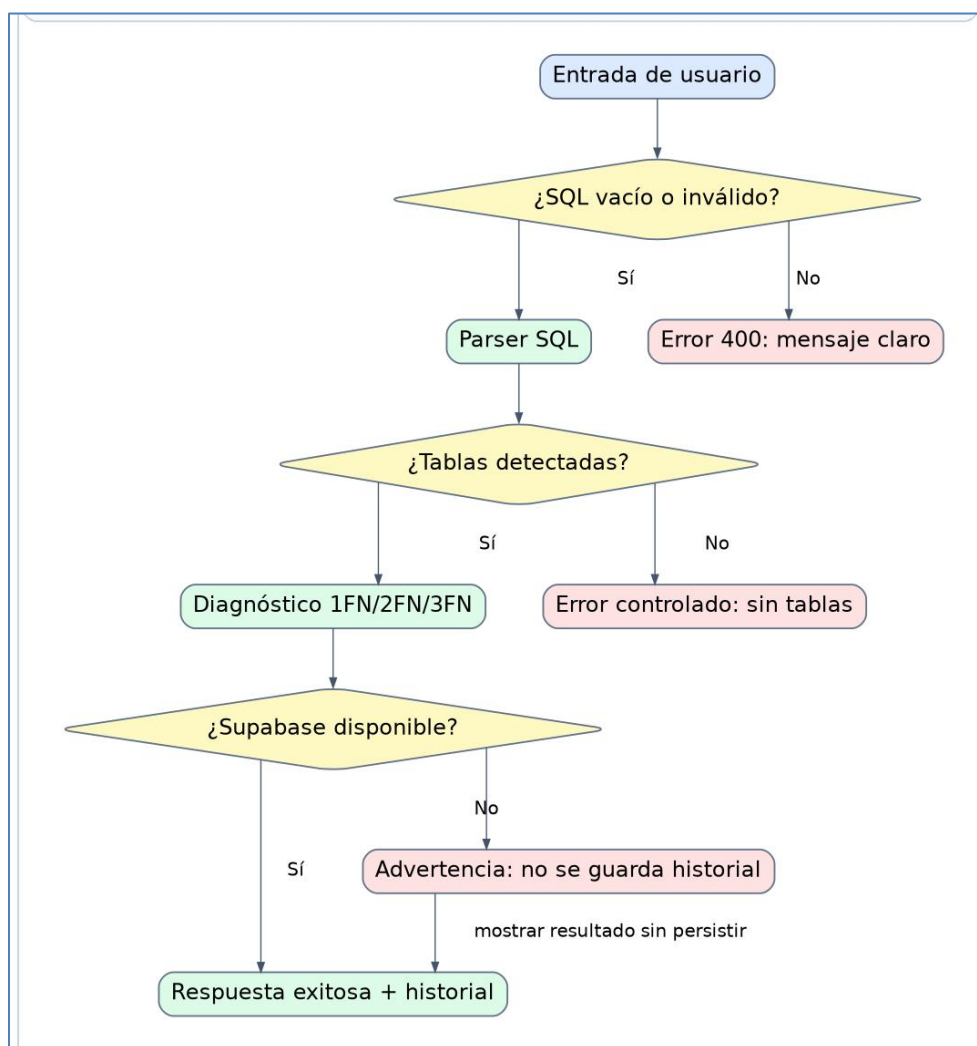


Figura 17. Diagrama de flujo de errores controlados.



VIII. VISTA DE DESARROLLO

A. Diagrama de capas

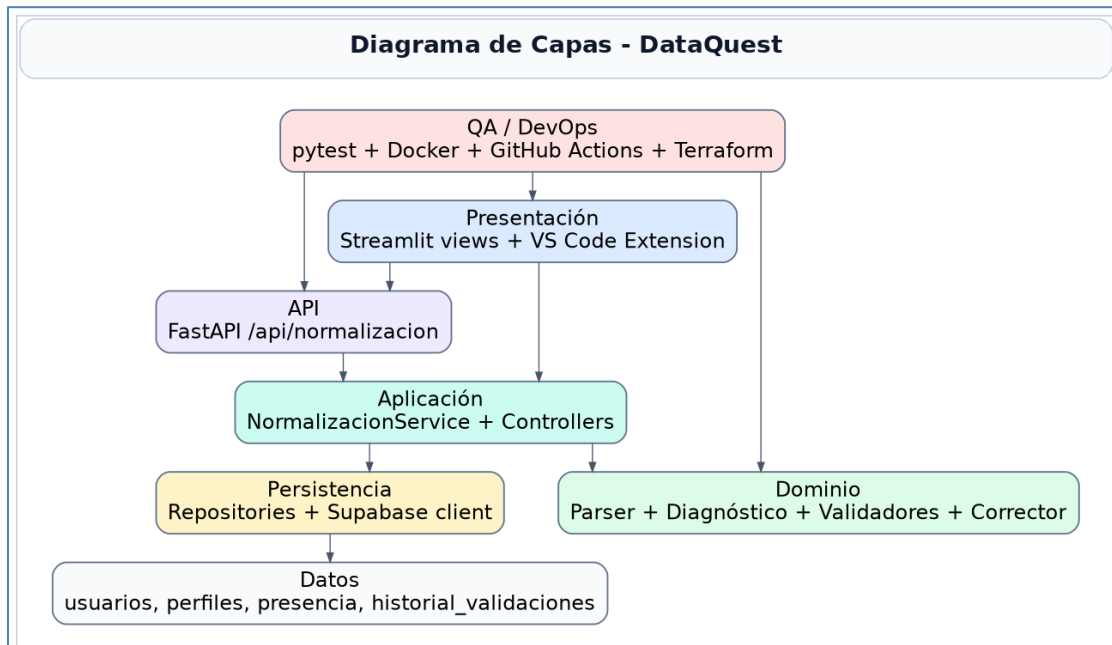


Figura 18. Diagrama de capas de DataQuest.

B. Diagrama de componentes

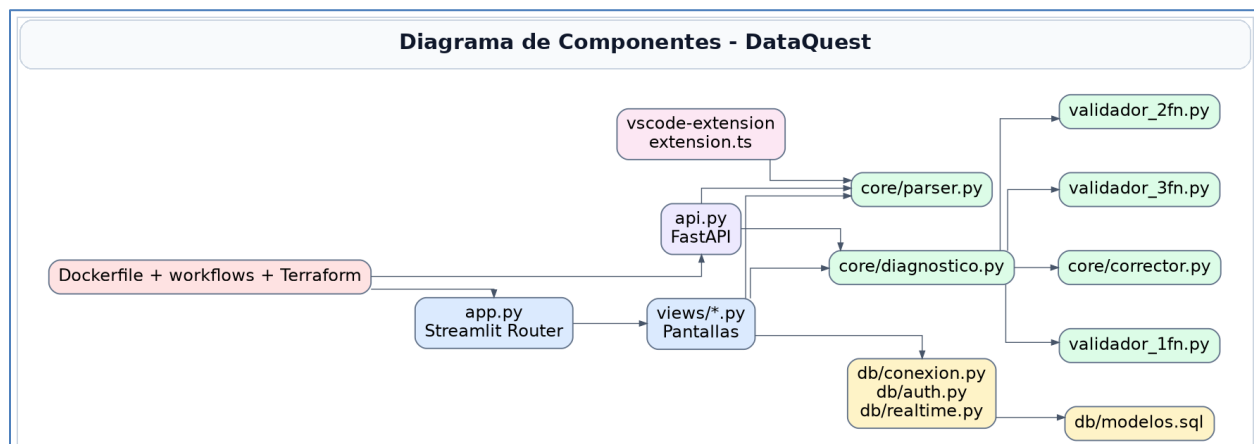


Figura 19. Diagrama de componentes de DataQuest.



C. Organización de carpetas

```
validador-normalizacion/  
├── app.py                # Interfaz Streamlit y navegación  
├── api.py                # FastAPI, endpoint POST /api/normalizacion  
├── controllers/          # Controladores de auth, comunidad, dashboard, validación  
├── core/                 # Parser, diagnóstico, validadores, corrector, CLI  
├── db/                   # Conexión, auth, realtime, modelos.sql  
├── views/                # Vistas Streamlit: auth, inicio, validador, resultados, historial  
├── visualizacion/        # Diagrama ER y schema visual  
├── tests/                # Pruebas unitarias, integración, UI, BDD  
├── vscode-extension/     # Extensión TypeScript para VS Code  
├── terraform/            # Infraestructura como código  
├── .github/workflows/    # CI/CD: tests, security, deploy  
└── Dockerfile            # Empaquetado para Streamlit
```

D. Dependencias técnicas

Dependencia	Uso arquitectónico	Estado
Python 3.10+ / 3.11	Lenguaje del core, CLI, Streamlit y API.	Implementado
Streamlit	Interfaz de usuario y navegación.	Implementado
FastAPI	Servicio externo de normalización.	Implementado
Supabase/PostgreSQL	Persistencia de perfiles, presencia e historial.	Implementado/parcial según entorno
Graphviz / Matplotlib	Visualización de diagramas ER y schema.	Implementado/parcial
Pytest / Playwright / BDD	Validación automatizada.	Implementado
Docker	Contenedor para Streamlit.	Implementado
GitHub Actions	Tests y seguridad.	Implementado
Terraform	Infraestructura propuesta.	Implementado como scripts
VS Code API	Extensión del editor.	Implementado

E. Contratos entre módulos

Origen	Destino	Entrada	Salida	Error posible
Streamlit	NormalizacionService	SQL, formato, FDs, nivel objetivo	ResultadoValidacion	Entrada vacía o SQL no parseable
FastAPI	NormalizacionService	RequestNormalizacion.sql	JSON success/error	HTTP lógico 400/500
ParserSQL	DiagnosticoService	Esquema parseado	Tablas, PKs, FKs y FDs	No detectar tablas
DiagnosticoService	Validadores	Tabla + dependencias	Violaciones por nivel	Dependencias incompletas
NormalizacionService	HistorialRepository	Resultado + JSONB	id_validacion	Supabase no disponible
VS Code Extension	CLI	Archivo temporal SQL + target NF	JSON y SQL corregido	Python/CLI no encontrado

F. API y endpoints

Método	Ruta	Entrada	Salida esperada	Estado
GET	/	Sin entrada	status, message, docs_url	Implementado
POST	/api/normalizacion	{"sql":"CREATE TABLE ..."} }	success, nivel_actual, violaciones_1fn/2fn/3fn, mejoras_opcionales, resumen	Implementado

Request ejemplo:

```
{  
  "sql": "CREATE TABLE alumno (id INT PRIMARY KEY, nombre TEXT, carrera_id INT, carrera_nombre TEXT);"  
}
```

Response esperado:

```
{  
  "success": true,  
  "nivel_actual": "2FN",  
}
```




```
"violaciones_1fn": [],  
"violaciones_2fn": [],  
"violaciones_3fn": [...],  
"mejoras_opcionales": [],  
"resumen": "El esquema global se encuentra en nivel: 2FN"  
}
```

IX. VISTA FÍSICA / DESPLIEGUE

La vista física muestra la distribución de DataQuest entre cliente, aplicación, API, motor Python y Supabase/PostgreSQL. El repositorio también incluye Dockerfile, Terraform y workflows de GitHub Actions, por lo que la arquitectura se documenta para ejecución local y despliegue académico.

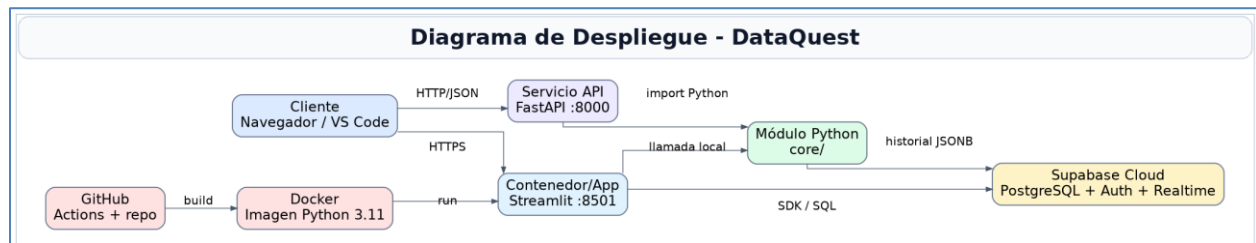


Figura 20. Diagrama de despliegue de DataQuest.

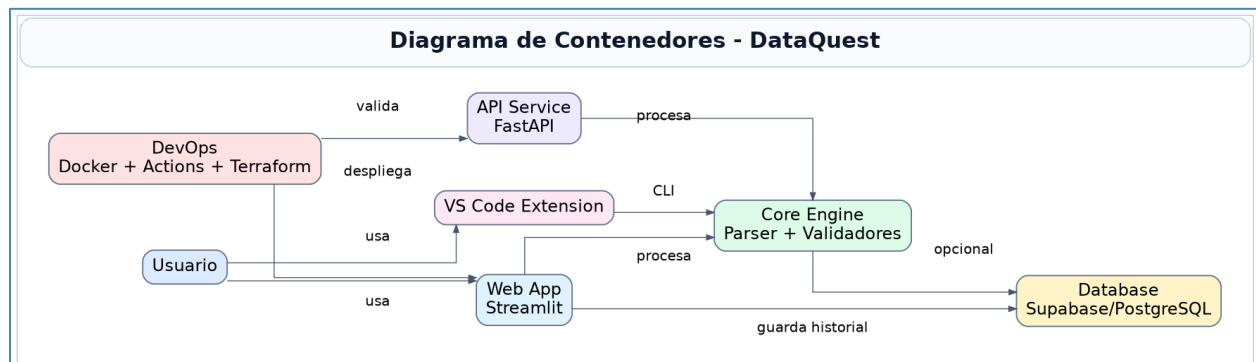


Figura 21. Diagrama de contenedores de DataQuest.

A. Infraestructura local

- Python 3.10+ o 3.11 instalado.
- Streamlit ejecutado mediante `streamlit run app.py`.
- FastAPI ejecutado mediante `uvicorn api:app` si se requiere servicio API.
- Variables `SUPABASE_URL`, `SUPABASE_KEY` y `DATABASE_URL` configuradas en entorno.
- Node.js instalado solo para pruebas o empaquetado de la extensión VS Code.

B. Infraestructura propuesta

- Streamlit empaquetado en contenedor Docker con puerto 8501.
- API FastAPI desplegable como servicio separado si el proyecto crece.



- Supabase/PostgreSQL como base administrada con RLS y Realtime.
- GitHub Actions para pruebas y seguridad antes de despliegue.
- Terraform como base de infraestructura reproducible.

C. Seguridad de despliegue

- Nunca subir claves reales al repositorio.
- Restringir CORS para producción; api.py lo mantiene abierto solo para desarrollo.
- Activar RLS en tablas sensibles y validar user_id en operaciones de historial.
- Usar HTTPS para consumo de API y conexión a Supabase.
- Respalidar historial_validaciones si el proyecto se usa en curso completo.

X. VISTA DE DATOS

La vista de datos se basa en cuatro tablas principales proporcionadas para el modelo de persistencia: usuarios, perfiles, presencia e historial_validaciones. Estas tablas permiten autenticar usuarios, almacenar perfil público, registrar actividad/presencia y conservar el historial completo de validaciones.

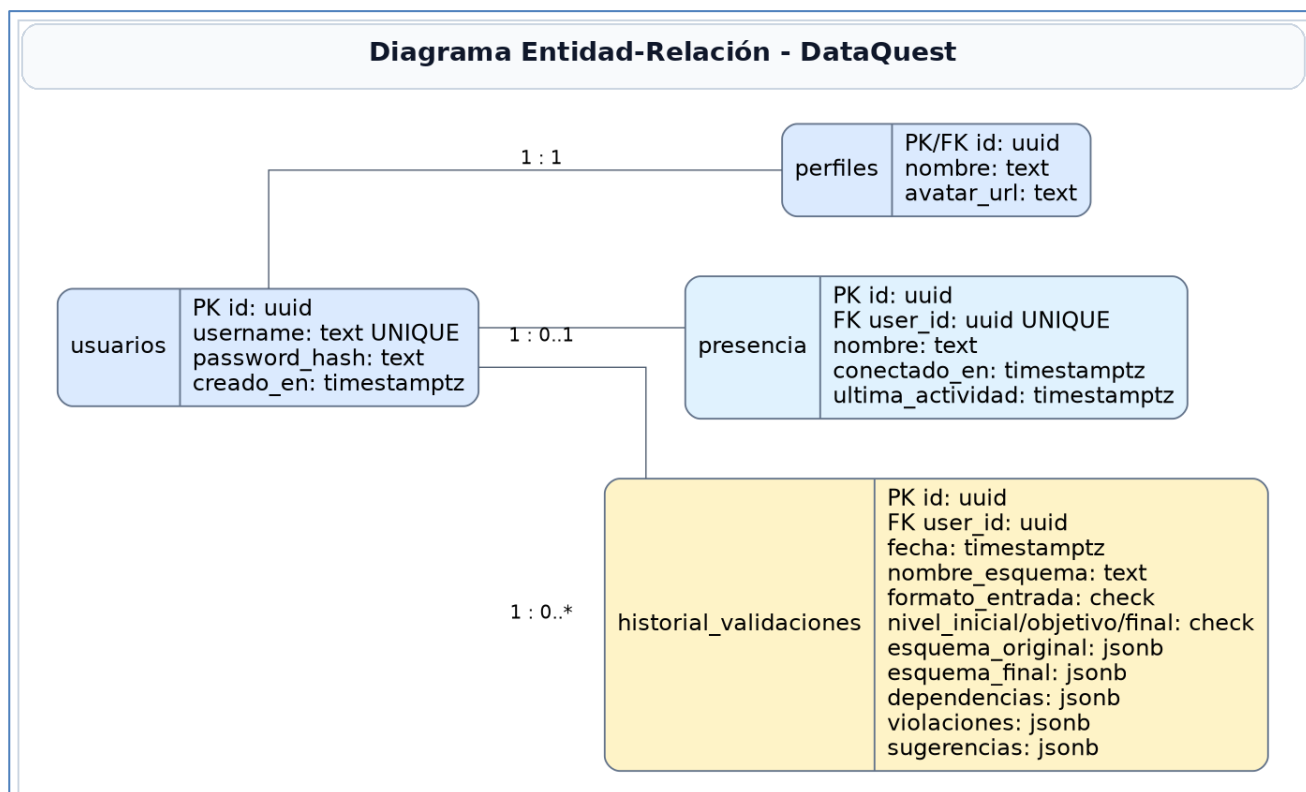


Figura 22. Diagrama entidad-relación de DataQuest.

A. Modelo lógico

Entidad	Descripción	Relación
---------	-------------	----------



usuarios	Identidad del usuario, nombre de acceso y hash de contraseña.	1 a 1 con perfiles, 1 a 0..1 con presencia, 1 a N con historial.
perfiles	Datos públicos del usuario: nombre y avatar.	Depende de usuarios mediante id.
presencia	Registro de conexión y última actividad.	Depende de usuarios mediante user_id único.
historial_validaciones	Registro de cada análisis con niveles, esquemas, dependencias, violaciones y sugerencias en JSONB.	Depende de usuarios mediante user_id.

B. Modelo físico

Tabla	PK	FK	Campos clave	Restricciones
usuarios	id uuid	-	username, password_hash, creado_en	username UNIQUE, password_hash NOT NULL
perfiles	id uuid	id -> usuarios.id	nombre, avatar_url	relación 1 a 1
presencia	id uuid	user_id -> usuarios.id	nombre, conectado_en, ultima_actividad	user_id UNIQUE
historial_validaciones	id uuid	user_id -> usuarios.id	nombre_esquema, formato_entrada, niveles, JSONB	CHECK de formato y niveles

C. JSONB y estructuras flexibles

DataQuest usa JSONB en historial_validaciones porque cada esquema SQL puede tener un número distinto de tablas, columnas, dependencias, violaciones y sugerencias. En un sistema académico esta decisión reduce complejidad inicial y permite registrar evidencia completa. La limitación es que parte de la integridad semántica queda en la aplicación, por lo que se recomienda validar estructura JSON antes de insertar y agregar índices GIN si el volumen crece.

Campos JSONB principales:

- esquema_original: estructura parseada antes de sugerencias.
- esquema_final: estructura posterior si se aplican mejoras.
- dependencias: dependencias funcionales ingresadas o inferidas.
- violaciones: resultados separados por 1FN, 2FN y 3FN.
- sugerencias: acciones recomendadas por el corrector.

D. Auditoría e historial

El historial es el núcleo de trazabilidad. Permite demostrar qué esquema se analizó, desde qué nivel partió, qué nivel objetivo se buscó, qué nivel final se obtuvo y qué violaciones/sugerencias sustentan el resultado. Arquitectónicamente, se recomienda añadir índices sobre user_id, fecha y eventualmente GIN sobre campos JSONB si se consultan diagnósticos en forma agregada.

XI. CALIDAD ARQUITECTÓNICA

Atributo	Estímulo	Artefacto	Respuesta arquitectónica	Medida de respuesta
Seguridad	Usuario no autenticado intenta ver historial.	Streamlit/Supabase	Denegar acceso y solicitar login.	Acceso bloqueado, sin fuga de registros.
Usabilidad	Usuario ingresa SQL vacío o inválido.	Streamlit/API	Mostrar mensaje claro y no ejecutar validadores.	Mensaje menor a 1 segundo.
Disponibilidad	Supabase no responde al guardar historial.	Web local	Mostrar resultado y advertir que no se guardó.	No se pierde la respuesta de validación.
Rendimiento	Usuario valida esquema mediano.	API/Core	Parser y validadores responden rápido.	Respuesta objetivo < 2 segundos.
Adaptabilidad	Se agrega BCNF.	Core	Crear nuevo validador sin cambiar UI/API.	Se mantiene contrato JSON.
Escalabilidad	Aumenta historial de usuarios.	PostgreSQL	Paginar e indexar user_id/fecha.	Consulta estable.
Mantenibilidad	Se modifica heurística de 3FN.	core/diagnostico.py	Pruebas de regresión.	Tests del core pasan.
Interoperabilidad	Cliente externo consume API.	FastAPI	Responder JSON estándar success/error.	Contrato documentado.



A. Observaciones de calidad por componente

El core es el componente más crítico porque contiene las reglas que determinan el valor académico del sistema. La API es crítica para interoperabilidad y debe endurecer CORS. La base de datos es crítica para trazabilidad y privacidad. La interfaz es crítica para usabilidad, por lo que los mensajes deben ser pedagógicos y no técnicos de bajo nivel.

XII. RIESGOS ARQUITECTÓNICOS

ID	Riesgo	Impacto	Probabilidad	Mitigación
RA-01	Parsing SQL incompleto	Alta	Media	Usar sqlparse/regex con casos de prueba y errores claros.
RA-02	Dependencias funcionales no detectadas	Alta	Media	Permitir ingreso manual de FDs y explicar heurísticas.
RA-03	JSONB sin validación	Media	Media	Validar esquema JSON desde aplicación antes de insertar.
RA-04	Credenciales expuestas	Alta	Baja	Variables de entorno, .env.example y .gitignore.
RA-05	CORS abierto en API	Media	Alta	Restringir allow_origins en producción.
RA-06	RLS mal configurado	Alta	Media	Probar políticas por usuario.
RA-07	Diagramas desactualizados	Media	Media	Actualizar FD04 cuando cambie código o BD.
RA-08	Interfaz confusa	Media	Media	Mensajes de validación claros y ejemplos.
RA-09	Falta de pruebas de regresión	Alta	Media	Mantener pytest y GitHub Actions.
RA-10	Historial crece sin paginación	Media	Media	Índices user_id/fecha y paginación.

XIII. CONCLUSIONES ARQUITECTÓNICAS

1. DataQuest presenta una arquitectura modular que separa interfaz, API, core de normalización, persistencia e integraciones, lo que mejora la mantenibilidad y la trazabilidad.
2. El modelo 4+1 permite justificar el sistema desde escenarios, lógica interna, procesos, desarrollo y despliegue, cubriendo los aspectos que un FD04/SAD debe demostrar.
3. El diagrama de clases diferencia entidades persistentes, servicios del dominio, controladores, repositorios y estructuras de salida, evitando confundir funciones reales con clases físicas no implementadas.
4. La tabla historial_validaciones y el uso de JSONB son decisiones adecuadas para conservar evidencia académica de esquemas, dependencias, violaciones y sugerencias.
5. El endpoint POST /api/normalizacion y la extensión VS Code amplían el valor arquitectónico del sistema al permitir consumo externo y uso desde herramientas de desarrollo.
6. La arquitectura requiere reforzar seguridad de CORS, validación JSONB, políticas RLS y pruebas de regresión para sostener calidad en evaluaciones exigentes.

XIV. RECOMENDACIONES

- Mantener sincronizados FD03, FD04, FD05, FD06, diccionario de datos, README y código fuente.
- Actualizar el diagrama de clases cada vez que se agreguen clases reales o servicios nuevos.
- Restringir CORS en FastAPI antes de cualquier despliegue público.
- Mantener RLS activo y probado en perfiles, presencia e historial_validaciones.
- Agregar validación estructural de JSONB antes de persistir resultados.
- Agregar índices para historial_validaciones(user_id, fecha DESC) y evaluar índices GIN sobre JSONB si el historial crece.



- Permitir ingreso manual de dependencias funcionales para reducir falsos positivos o negativos en 2FN y 3FN.
- Documentar OpenAPI/Swagger con ejemplos reales de request y response.
- Mantener pruebas unitarias del core y pruebas de integración API en GitHub Actions.
- Exportar diagramas en alta resolución y conservar sus fuentes editables para correcciones del docente.

ANEXO ÚNICO. FUENTES EDITABLES RESUMIDAS DE DIAGRAMAS

Se incluye un resumen editable de los principales diagramas para que puedan ser reconstruidos o ajustados si el docente solicita cambios. Los diagramas del documento fueron generados como imágenes de alta resolución con fuente integrada.

Ejemplo PlantUML - Clases principales:

```
class Usuario {
    +id: UUID
    +username: text
    +password_hash: text
}
class HistorialValidacion {
    +id: UUID
    +nombre_esquema: text
    +esquema_original: JSONB
    +violaciones: JSONB
    +sugerencias: JSONB
}
class ParserSQL { +parse_schema(); +parse_sql(); +parse_fds() }
class DiagnosticoService { +diagnosticar_esquema(); +inferir_dependencias_heuristica() }
class Validador1FN { +validar_1fn() }
class Validador2FN { +validar_2fn() }
class Validador3FN { +validar_3fn() }
Usuario "1" -- "0..*" HistorialValidacion
DiagnosticoService --> Validador1FN
DiagnosticoService --> Validador2FN
DiagnosticoService --> Validador3FN
ParserSQL --> DiagnosticoService
```

Ejemplo PlantUML - Secuencia API:

```
APIConsumer -> FastAPI: POST /api/normalizacion(sql)
FastAPI -> RequestModel: validar campo sql
FastAPI -> NormalizacionService: procesar esquema
NormalizacionService -> ParserSQL: parse_schema()
NormalizacionService -> DiagnosticoService: diagnosticar_esquema()
DiagnosticoService -> Validador1FN: validar_1fn()
DiagnosticoService -> Validador2FN: validar_2fn()
DiagnosticoService -> Validador3FN: validar_3fn()
NormalizacionService -> APIConsumer: JSON success/error
```